



NAMIBIA
UNIVERSITY
OF SCIENCE AND
TECHNOLOGY

PAP521S – PROGRAMMING IN PRACTICE

PROJECT A: MUNICIPAL FINANCIAL MANAGEMENT SYSTEM

Group members:

Siteketa Marthin 225030969

Fiindje Mate 223083593

Abigail Da Cunha 226074943

Enerist Shilumbu 222093951

Kayoko Manguluki 226074404

Dipundhi Paul Peter 225058146

Tavey Ngenokesho 226008053

1. Introduction

The Municipal Financial Management System (MFMS) was developed as a basic system for managing important financial and administrative information within a municipality. The primary aim of this project is to utilize the knowledge about the C language programming learned in the early stages of the PAP521S course into a real-life situation.

The local authority faces handling huge amounts of data on a daily basis, such as employee data, budgets for departments, suppliers, and municipal assets. Handling all this data manually would make it tough to look up particular data or do any calculations or preparation of reports. The system developed for this project provides a simple way of keeping this information organized in one application.

The project was developed using ANSI C (C99), with Visual Studio Code and GCC being used as the development environment. Git and GitHub were also used for version control and to help group members work together. The system is not intended to be a complete professional municipal management system. Instead, Project A provides a foundation that can be improved and expanded in Project B.

2. Problem Description

The administration in a municipal government has to handle huge volumes of financial and administrative information. Such pieces of information include details about personnel and salaries, budgetary allocations, expenditures, suppliers, and assets of the municipal government. However, when there is no proper organization of such pieces of information, it becomes hard to locate information and make accurate computations.

This is because, in handling budgets of various departments, the municipal administration must know the total budgetary allocation, the amount of money that has been used up and how much remains within each of the departments. Furthermore, the municipal administration must be able to tell which departments have overspent their budgetary allocations.

Supplier information also needs to be kept properly. This may include the supplier's name, contact details and location. In the same way, employee records and information about municipal assets need to be stored in a way that makes them easy to search and display.

The MFMS was developed to address these issues by providing separate sections for employee management, budget management, supplier management, asset management and reporting. These sections can all be accessed through the main menu, making it easier for the user to move between the different parts of the system.

3. System Objectives

The main objective of the project was to develop a functional, menu-driven C application that demonstrates the programming concepts covered during Weeks 1–8 of the course.

The specific objectives of the system are to:

1. Design a simple main menu.
2. Users should be able to enter and display the details of employees.
3. Allow the user to find employees by searching for them.

4. Calculate and show details about the salary of the employees.
5. Enter and manage budget details of the departments.
6. Calculate the spending and balance of the budget.
7. Find out which departments have exceeded their budgets.
8. Manage supplier details.
9. Prepare a simple record of municipal assets.
10. Search for assets and show the information.
11. Prepare simple reports using the information in the program.
12. Use arrays, strings, loops, conditionals and functions properly.
13. Validating user inputs and preventing invalid inputs.
14. The program should be modularized, that is, not everything should be written in the main() function.
15. GitHub will be used to enable team collaboration and track individual efforts.

The above mentioned objectives are derived from the requirements of the project and help us show the use of variables, operators, decision making statements, loops, arrays, strings and functions.

4. System Features

The MFMS is made up of several main features that each deal with a different type of municipal information.

4.1 Main Menu

The main menu is the starting point of the system. It gives the user access to the different modules of the program. The main options include Employee Management, Budget Management, Supplier Management, Asset Management, Reports and Exit.

The menu uses decision-making structures such as switch statements or if statements to determine which option the user has selected. This makes it possible for the user to move between the different sections of the system

4.2 Employee Management

The employee management section is used to store and manage employee records. The information stored for each employee includes the Employee ID, employee name, department, basic salary, housing allowance and transport allowance.

Users can add new employees, display existing employee records, search for a particular employee and calculate salary information. Arrays are used to store multiple employee records, while loops are used to go through the records when displaying or searching for employees.

4.3 Budget Management

The budget management section is used to keep track of departmental budgets. The user can enter the department name, allocated budget and expenditure.

The system then calculates the remaining budget using the following formula:

$\text{Remaining Budget} = \text{Allocated Budget} - \text{Expenditure}$

The program also checks whether a department is still within its allocated budget or whether it has spent more than the amount that was allocated to it. This section demonstrates the use of arithmetic operators, conditions and functions.

4.4 Supplier Management

The supplier management section stores basic information about suppliers that provide goods or services to the municipality. The information includes the Supplier ID, supplier name, email address, telephone number and town or location.

Users can add suppliers, display registered suppliers and search for supplier information. String functions such as strcmp() can be used to compare names or search for a specific supplier.

4.5 Asset Management

The asset management section provides a basic register of municipal assets. Some examples of assets that can be recorded include vehicles, computers, buildings, equipment and office furniture.

Each asset can contain information such as its ID, name, type, purchase value, department and condition. Users can display the registered assets and search for a particular asset when needed.

4.6 Reports

The reporting section provides summary information based on the records stored in the system.

For employees, the report can show information such as the total number of employees, average salary, highest salary and lowest salary. The budget report can show the total allocated budget, total expenditure, remaining budget and the departments that have gone over their allocated budgets.

Reports can also be generated for suppliers and municipal assets, giving the user a quick overview of the information stored in the system.

5. Program Design

The program has been developed as a menu-based program. In order to avoid having the entire program's code contained in one big main() function, we have divided the program into different functions. Every function performs a certain task and this makes it more manageable and modifiable.

Some of the main functions used or planned for the system include:

Menu and Control

```
void displayMenu();
```

Employee Management

```
void addEmployee();
```

```
void displayEmployees();
```

```
void searchEmployee();
```

```
void calculateSalary();
```

Supplier Management

```
void addSupplier();
```

```
void displaySuppliers();
```

```
void searchSupplier();
```

Budget Management

```
void calculateBudget();
```

```
void displayBudget();
```

Asset Management

```
void addAsset();
```

```
void displayAssets();
```

```
void searchAsset();
```

Reports

```
void displayReports();
```

The general flow of the system can be described as:

Input → Processing → Storage → Search → Calculation → Output

The user first enters information into the system. The software then analyses and saves this data. In case there is a need to retrieve any data or do some calculations, the output will be shown on the screen.

6. Challenges Encountered

During the development of the system, we came across a number of challenges.

Function Organization:

Because of the existence of multiple modules in the system, it was crucial that each function have a defined role. It was equally important that the functions do not interfere with each other in their proper functioning.

Input Handling:

The user can sometimes provide invalid information, for example, a negative salary, a negative budget, or an invalid option on the menu. This implied that input validation should be taken into consideration while creating the program.

Searching Records:

The system needs to find specific records using information such as employee IDs, supplier names or asset IDs. This required us to compare the user's input with the records already stored in the arrays.

Array Management:

Storing several records in arrays and making sure that the correct record is accessed required careful use of indexes and loops.

Group Coordination:

Because different group members worked on different sections of the system, combining the code was another challenge. The different sections had to be checked and adjusted so that they could work together as one program.

7. Solutions Implemented

In order to solve this problem, the project had to be broken down into several functions that performed specific operations. This approach simplified code understanding and debugging in case any issues occurred.

Input validation was also added to prevent obviously incorrect information from being entered, such as negative salaries or invalid menu choices. Arrays and loops were used to store and manage multiple records, while search functions were used to compare the information entered by the user with the existing records.

GitHub was used to help the group manage changes to the code and combine the different sections developed by each member. The individual sections were tested before they were integrated into the main system. This helped to identify problems early and made the final integration easier.

8. Conclusion

The Municipal Financial Management System provides a basic way of managing important municipal financial and administrative information. It brings together employee management, budget management, supplier management, asset management and reporting in one menu-driven C application.

Working on this project gave us the opportunity to put the programming concepts covered during the first part of PAP521S into practice. These included variables, data types, operators, conditions, loops, arrays, strings and functions.

The development of the system also showed us the importance of organizing code properly, checking user input, testing different sections and working together as a group. Although Project A is only a foundation, it provides a good starting point for Project B, where additional and more advanced features can be added to the system.